

2.1 RRCLite Buzzer Source Code Analysis

1. Program Logic

This section mainly focuses on controlling the sound of the buzzer on the STM32 main control board.

2. Hardware Connection

The hardware connection diagram is as below:



3. Program Download

The example for this section is "RosRobotControllerLite_ros".

For specific instructions on how to download the program, refer to the document: "2.STM32 Development Fundamentals/3.Project Compilation and Download".

4. Program Outcome

After powered on, the buzzer will first sound for 1 second.

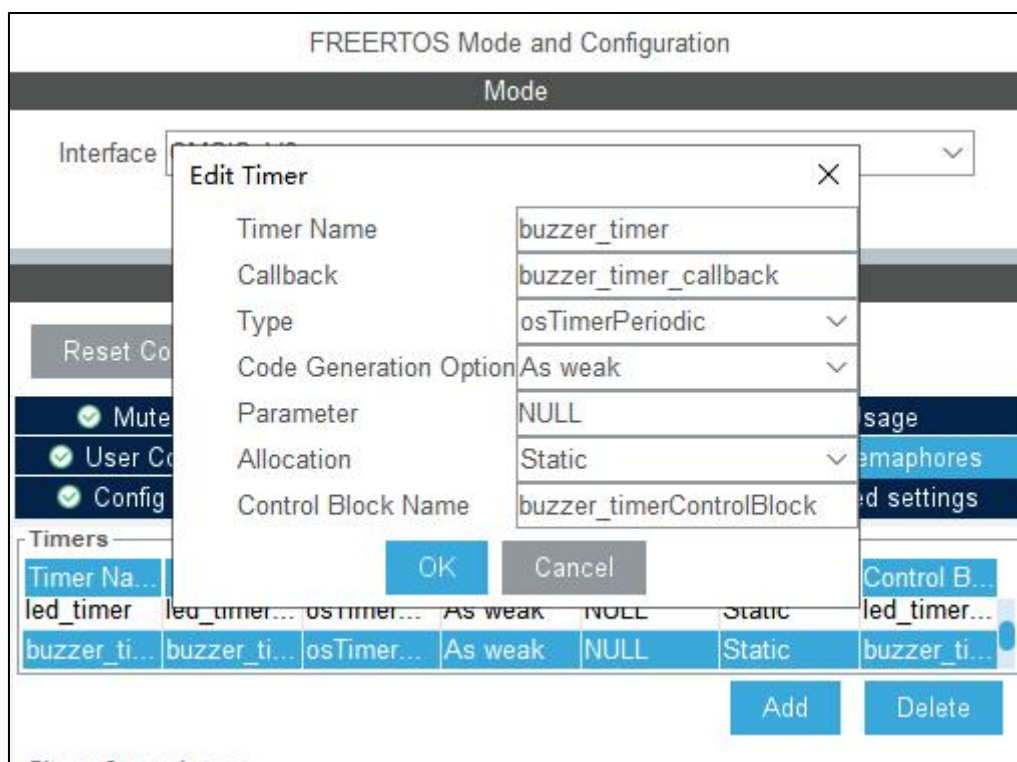
Then, it will sound three beeps at a frequency of 1500 Hz. The duration of each beep is 50 milliseconds, and the interval between two beeps is 100 milliseconds.

After that, it will sound three beeps at a frequency of 2000 Hz. The duration of each beep is 50 milliseconds, and the interval between two beeps is 100 milliseconds.

Finally, the buzzer will be turned off.

5. State Machine Configuration

Use STM32CubeMX to check the information on software timers and state machines. In the “Pinout & Configuration” section, go to “FREERTOS” and open “Timers and Semaphores”. You will find the details of “buzzer_timer” there.



The current application is in Hiwonder/Peripherals/led.c.

6. Source Code Analysis

The source code for this course is located in the same folder as this tutorial document, mainly analyzing the files Hiwonder/System/app.c and Hiwonder/Peripherals/led.c.

6.1 Buzzer Functionality Implementation

```
void app_task_entry(void *argument)
{
    //buzzer handle
    extern osTimerId_t buzzer_timerHandle;
    /* hardware initialization */
    buzzers_init(); //buzzer initialization
    //Start the buzzer timer to let it operate in the interrupt, and call the interface function later
    //Parameter 1: Timer handle, Parameter 2: Timer working interval in ms
    osTimerStart(buzzer_timerHandle, BUZZER_TASK_PERIOD);

    buzzer_on(buzzers[0], 1000); // enable the buzzer to sound for 1 second
    osDelay(1000); // after delay for 1 second

    buzzer_didi(buzzers[0], 1500, 50, 100, 3);
    // 1500 indicates the frequency
    // The duration of each beep is 50ms and the interval between two beeps is 100ms
    // The last parameter specifies the number of times the sound sequence should be repeated. In this
    osDelay(1000);

    buzzer_didi(buzzers[0], 2000, 50, 100, 3); // Everything is the same except for the frequency
    osDelay(1000); // After delaying for 1 second, close the buzzer
    buzzer_off(buzzers[0]); // close

    // Loop: The loop in the RTOS task must have osDelay or other system blocking functions, otherwise it
    for(;;) {
        osDelay(300);
    }
}
```

Above is the App.c file. “app_task_entry()” serves as the user's entry function.

The main programming flow is as follows:

An externally defined variable buzzer_timerHandle is used as the timer handle.
The buzzers_init function is called to initialize the buzzer.
Then, the osTimerStart function is used to start the timer, passing BUZZER_TASK_PERIOD as the timer's working interval. This enables the buzzer control function to be called in the timer interrupt.

The program uses a timer callback function to control the sound of the buzzer. Delay functions are used to control the duration and the number of times of the buzzer sound.

```

/* Create the mutex(es) */
/* creation of oled_mutex */
oled_mutexHandle = osMutexNew(&oled_mutex_attributes);

/* USER CODE BEGIN RTOS_MUTEX */
/* add mutexes, ... */
/* USER CODE END RTOS_MUTEX */

/* Create the semaphores(s) */
/* creation of packet_tx_idle */
packet_tx_idleHandle = osSemaphoreNew(1, 1, &packet_tx_idle_attributes);

/* creation of packet_rx_not_empty */
packet_rx_not_emptyHandle = osSemaphoreNew(1, 1, &packet_rx_not_empty_attributes);

/* creation of IMU_data_ready */
IMU_data_readyHandle = osSemaphoreNew(1, 1, &IMU_data_ready_attributes);

/* creation of spi_tx_finished */
spi_tx_finishedHandle = osSemaphoreNew(1, 1, &spi_tx_finished_attributes);

/* creation of serial_servo_rx_complete */
serial_servo_rx_completeHandle = osSemaphoreNew(1, 1, &serial_servo_rx_complete_attributes);

/* USER CODE BEGIN RTOS_SEMAPHORES */
/* add semaphores, ... */
/* USER CODE END RTOS_SEMAPHORES */

/* Create the timer(s) */
/* creation of button_timer */
button_timerHandle = osTimerNew(button_timer_callback, osTimerPeriodic, NULL, &button_timer_attributes);

/* creation of led_timer */
led_timerHandle = osTimerNew(led_timer_callback, osTimerPeriodic, NULL, &led_timer_attributes);

/* creation of buzzer_timer */
buzzer_timerHandle = osTimerNew(buzzer_timer_callback, osTimerPeriodic, NULL, &buzzer_timer_attributes);

```

(In the freertos.c file, the timer handle is associated with the corresponding timer callback function)

```

58
59
60 /**
61  * @brief System timer callback function
62  * @details Refreshes LED status at a set interval, which is configured through CubeMx
63  * @retval None.
64  */
65
66 void buzzer_timer_callback(void *argument)
67 {
68     buzzer_task_handler(buzzers[0], BUZZER_TASK_PERIOD);
69 }
70
71

```

(In the buzzer_porting.c file, the timer callback function for the buzzer executes the buzzer task by default)

6.2 Control Buzzer On, Off and Sound Times

```
int buzzer_on( BuzzerObjectTypeDef *self, uint32_t freq)
{
    BuzzerCtrlTypeDef ctrl_structure = {
        .freq = freq,
        .ticks_on = 1,
        .ticks_off = 0, /* If the silent time is 0, the buzzer makes a long sound */
        .repeat = 0,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}

int buzzer_off(BuzzerObjectTypeDef *self)
{
    BuzzerCtrlTypeDef ctrl_structure = {
        .freq = 0,
        .ticks_on = 0, /* If the sound time is 0, it is silent */
        .ticks_off = 0,
        .repeat = 0,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}
```

```
int buzzer_didi(BuzzerObjectTypeDef *self, uint32_t freq, uint32_t ticks_on, uint32_t ticks_off, uint32_t repeat)
{
    BuzzerCtrlTypeDef ctrl_structure = {
        .freq = freq,
        .ticks_on = ticks_on,
        .ticks_off = ticks_off,
        .repeat = repeat,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}

void buzzer_object_init(BuzzerObjectTypeDef *self)
{
    self->stage = BUZZER_STAGE_IDLE;
}
```

Above is the buzzer.c file. It constructs a buzzer object and provides three methods for controlling the buzzer. These functions can be called to turn the buzzer on/off and set its sound frequency.

In these functions, different members for controlling the buzzer structure (working frequency, sound duration, mute time, repetition times) are assigned different values, based on different functions. The defined structure is then passed into the buzzer message queue. The data will be retrieved and processed from other functions.

The buzzer_on_() function is a structure for turning the buzzer on.

The buzzer_off() function is a structure for turning the buzzer off.

The buzzer_didi function is a structure for controlling the buzzer to sound multiple times.

The buzzer_object_init(LEDObjectTypeDef *self) function initializes the buzzer object. In this function, the state of the state machine is set to BUZZER_STAGE_IDLE.

When the buzzer state machine is in the idle mode, each time the buzzer task

triggered by the buzzer timer will not perform any operation.

6.3 Physical Implementation of Buzzer Operating Parameter

```

1  /**
2  * @file buzzer.c
3  * @author Lu Yongping (Lucas@hiwonder.com)
4  * @brief Implement hardware & system-independent buzzer control
5  * @version 0.1
6  * @date 2023-05-18
7  *
8  * @copyright Copyright (c) 2023
9  *
10 */
11 #include "buzzer.h"
12
13 void buzzer_task_handler(BuzzerObjectTypeDef *self, uint32_t period)
14 {
15     /* Attempt to retrieve new control data from the queue. If it is successful, reset the state machine and start a new control loop */
16     if(self->get_ctrl_block(self, &self->ctrl_structure) == 0) {
17         self->stage = BUZZER_STAGE_START_NEW_CYCLE;
18     }
19     /* State machine processing */
20     switch(self->stage) {
21         case BUZZER_STAGE_START_NEW_CYCLE: {
22             if(self->ctrl_structure.ticks_on > 0 && self->ctrl_structure.freq > 0) {
23                 self->set_pwm(self, self->ctrl_structure.freq); /* buzzer sounds */
24                 if(self->ctrl_structure.ticks_off > 0) { /* If the silence time is not 0, the buzzer beeps; otherwise, it sounds continuously */
25                     self->ticks_count = 0;
26                     self->stage = BUZZER_STAGE_WAITING_OFF; /* wait for the sound duration to end */
27                 } else {
28                     self->stage = BUZZER_STAGE_IDLE; /* Sound continuously and change to the idle state */
29                 }
30             } else { /* If the silent time is 0, the buzzer makes a long sound */
31                 self->set_pwm(self, 0);
32                 self->stage = BUZZER_STAGE_IDLE; /* be silent and transite to the idle state */
33             }
34             break;
35         }
36         case BUZZER_STAGE_WAITING_OFF: {
37             self->ticks_count += period;
38             if(self->ticks_count >= self->ctrl_structure.ticks_on) { /* End the sound duration */
39                 self->set_pwm(self, 0);
40                 self->stage = BUZZER_STAGE_WAITING_PERIOD_END;
41             }
42             break;
43         }
44     }
45 }

```

```

void buzzer_task_handler(BuzzerObjectTypeDef *self, uint32_t period)
{
    /* Attempt to retrieve new control data from the queue. If it is successful, reset the state machine and start a new control loop */
    if(self->get_ctrl_block(self, &self->ctrl_structure) == 0) {
        self->stage = BUZZER_STAGE_START_NEW_CYCLE;
    }
}

```

Above is the buzzer.c file. It constructs a buzzer object and controls the buzzer based on the different buzzer state machines. When new data is retrieved from the buzzer message queue, the state machine is set to the beginning of a new work cycle. The buzzer is controlled accordingly.

The new data obtained from the buzzer message queue are exactly the data stored in functions such as buzzer_on and buzzer_didi.